



IHK-Ausbildungsnachweise

Desktop-Anwendung – Kurzdokumentation

Version 0.1.0 | Stand: 10. Juni 2026

Die Anwendung unterstützt Fachinformatiker-Azubis beim Führen ihrer wöchentlichen Ausbildungsnachweise. Stichworte werden links eingetragen, eine *lokale* Sprachmodell-Engine strukturiert sie thematisch, und rechts erscheint das daraus erzeugte PDF.

1 Hintergrund & Aufbau

Die App ist in Python mit **PySide6** (Qt 6) umgesetzt und in klar getrennte Module gegliedert:

- `ihk_nachweise/app.py` – Anwendungseinstieg (Qt-Setup).
- `ihk_nachweise/main_window.py` – Orchestrierung von Eingabe, LLM, PDF-Erzeugung und Anzeige.
- `ihk_nachweise/widgets/` – Oberfläche: Metadaten-Leiste (oben), Eingabe-Panel (links), PDF-Anzeige via `QPdfView` (rechts).
- `ihk_nachweise/llm/` – Modellkatalog, Download- und Generierungs-Worker.
- `ihk_nachweise/pdf/` – PDF-Erzeugung mit **reportlab**.
- `ihk_nachweise/storage/` – Rohdaten als `.txt` (verlustfrei wieder einlesbar).
- `ihk_nachweise/config.py`, `paths.py` – Konfiguration (JSON) und plattformübliche Speicherorte.

Langlaufende Aufgaben (Modell-Download, Textgenerierung) laufen in eigenen Qt-Threads, damit die Oberfläche bedienbar bleibt.

2 Lokale LLM

Die Sprachverarbeitung erfolgt **vollständig lokal** – es werden keine Daten an einen Online-Dienst gesendet.

- **Engine:** `llama-cpp-python` (nutzt Metal auf macOS, CPU unter Windows). Die nativen Bibliotheken sind im Paket enthalten.
- **Modelle:** GGUF-Dateien von Hugging Face. Auswählbar u. a. *Qwen2.5 3B* (Standard, ca. 1,9 GB), *Llama 3.2 3B*, *Qwen2.5 7B* sowie *Qwen2.5 1.5B*.
- **Download bei Bedarf:** Das gewählte Modell wird beim ersten Verwenden automatisch heruntergeladen (mit Fortschrittsanzeige) und danach wiederverwendet. Es ist *nicht* im Installationspaket enthalten.
- **Strukturierung:** Aus den Stichworten werden wenige thematische Oberkategorien mit Überschriften und Stichpunkten gebildet.
- **Neu generieren:** Erzeugt das Ergebnis mit mehr Variation neu und überschreibt die vorige Fassung.

3 Installation & Distribution

Entwicklung (aus dem Quellcode)

```
.venv/bin/pip install -r requirements.txt
```

```
.venv/bin/python main.py
```

Eigenständige Pakete

Mit **PyInstaller** werden gebündelte Pakete erzeugt, die alle Bibliotheken enthalten:

```
.venv/bin/pip install pyinstaller
```

```
# macOS (Apple Silicon) -> dist/IHKNachweise.dmg
bash packaging/build_macos.sh
```

```
# Windows 11 -> dist/IHKNachweise/IHKNachweise.exe
packaging\build_windows.bat
```

Das App-Icon (packaging/icon.svg) wird über packaging/make_icons.py in .icns (macOS) und .ico (Windows) umgewandelt und vom Build automatisch eingebunden. Ein GitHub-Actions-Workflow (.github/workflows/release.yml) baut beide Plattformen, sobald ein Tag **v*** gepusht wird.

4 Speicherorte / Pfade

Die Anwendung ist **portabel**: Alle Daten liegen in einem Ordner **Daten** *neben* der Anwendung. Es werden **keine Administratorrechte** benötigt und nichts in geschützte System- oder Programmdordner geschrieben. Die App darf in einem beliebigen, beschreibbaren Ordner liegen (Home, Netzlaufwerk, USB-Stick) und wird manuell gestartet.

```
<frei gewählter Ordner>/
  IHKNachweise(.exe / .app)      # die Anwendung
  Daten/
    config.json                 # Konfiguration
    modelle/                   # heruntergeladene GGUF-Modelle
    Nachweise/                 # erzeugte PDFs + Rohdaten (.txt)
```

Auf macOS wird bewusst *neben* das **.app**-Bundle geschrieben (nicht hinein), um dessen Signatur nicht zu verletzen. Im Entwicklungsbetrieb (python main.py) entsteht **Daten/** in der Projektwurzel.

Umleitbar: **IHK_DATA_DIR** verlegt den gesamten Datenordner an einen beliebigen Ort; **IHK_MODELS_DIR** und **IHK_CONFIG_DIR** verlegen gezielt nur Modell- bzw. Konfigurationsordner.

Hinweis: Beim Aktualisieren der App den Ordner **Daten** behalten – er enthält Modell, Konfiguration und alle Nachweise.

5 Laufender Betrieb

1. **Metadaten** oben setzen: Name, Kalenderwoche (mit überschreibbarem Datumsbereich – wichtig bei Arbeitsbereichswechsel innerhalb einer Woche), Arbeitsbereich, Betreuer/in, Modell sowie Speicherort und Dateinamen-Muster.
2. **Stichworte** der Woche links eintragen.
3. **„Erzeuge“** klicken: Bei Bedarf wird das Modell geladen, anschließend strukturiert die LLM die Eingaben und das PDF erscheint rechts.
4. **Ergebnis prüfen.** Ist es unbefriedigend, erzwingt „Neu generieren“ eine neue Fassung.
5. **Ausgabe:** Pro Nachweis entstehen eine **.pdf** und eine gleichnamige **.txt** (Rohdaten). Die Fußzeile des PDF nennt das verwendete Modell sowie die Erzeugungsdaten.

6. **Später fortsetzen:** Über „Rohdaten laden“ lässt sich eine `.txt` wieder einlesen und ein neues PDF erzeugen.

Die Metadaten (Name, Listen für Arbeitsbereiche/Betreuer, Pfade, Muster, zuletzt gewähltes Modell) werden in der Konfigurationsdatei gespeichert und beim nächsten Start wiederhergestellt; sie lassen sich über das „Datei“-Menü zurücksetzen.